

Natív Programozás ZH

SZTE Szoftverfejlesztés Tanszék
2025. őszi félév

Technikai ismertető

- A programot C++ nyelven kell megírni.
- A megoldást a *Bíró* fogja kiértékelni.
 - A Feladat beadása felületen a Feltöltés gomb megnyomása után ki kell várni, amíg lefut a kiértékelés. **Kiértékelés közben nem szabad az oldalt frissíteni vagy a Feltöltés gombot újból megnyomni** különben feltöltési lehetőség veszik el!
- Feltöltés után a *Bíró* a programot g++ fordítóval és a
-std=c++20 -Wall -Werror -static -O2 -DTEST_BIRO=1
paraméterezéssel fordítja és különböző tesztesetekre futtatja.
- A program működése akkor helyes, ha a tesztesetek futása nem tart tovább 5 másodpercnél és hiba nélkül (0 hibakóddal) fejeződik be, valamint a program működése a feladatkiírásnak megfelelő.
- A *Bíró* által a `riport.txt`-ben visszaadott lehetséges hibakódok:
 - Futási hiba 6: Memória- vagy időkorlát túllépés.
 - Futási hiba 8: Lebegőpontos hiba, például nullával való osztás.
 - Futási hiba 11: Memória-hozzáférési probléma, pl. tömb-túlinde克斯, null pointer használat.
- A beadások számát és a beadási határidőt a bíróban lehet megtekinteni.
- A programban a `#ifndef TEST_BIRO` és a hozzá tartozó `#endif` közötti rész nem lesz figyelembe véve a kiértékelés során.
- A bíróba egyetlen .cpp forrásfájlt lehet feltölteni (zip fájlt nem fogja kiértékelni).
- Technikai hiba esetén a technikai hiba elhárulása után a hallgató folytathatja a ZH-t, és a ZH határidejét a hibának megfelelően módosítjuk. Ha a hallgató a technikai hiba miatt nem képes folytatni a ZH-t, akkor az adott alkalom igazolt hiányzásnak minősül.
- A ZH megírásához hivatalos fényképes igazolvány szükséges. Ennek hiányában a ZH nem teljesíthető.

Általános követelmények, tudnivalók

- A programnak követnie kell az objektum-orientált programozás elveit!
- Csak a leírásban szereplő osztályokat, metódusokat és adattagokat kell megvalósítani, egyéb dolgokért nem jár plusz pont.
- Minden metódus, amelyik nem változtatja meg az objektumot, legyen konstans! Ha a paramétert nem változtatja a metódus, akkor a paraméter legyen konstans!
- string összehasonlításoknál az egyezés a pontos egyezést jelenti, azaz ha kis-nagy betűben térnek el, akkor már nem tekinthetők egyenlőnek (pl. a "piros" != "Piros")
- A leírásokban bemutatott példákban a stringek köré rakott idézőjelek nem részei az elvárt kimenetnek, azok csak a string határait jelölik. Például ha az szerepel, hogy a példa bemenetre az elvárt kimenet az, hogy "3 alma", akkor az elvárt kimenet idézőjelek nélkül az 3 alma, de a szóköz szükséges!
 - A tesztesetekben nem lesz ékezetes szöveg kiírása.
- Az elvárt kimeneteknek karakterről karakterre olyan formátumúnak kell lennie, ami a feladatban le van írva (szóközöket és sortöréseket is beleértve).
- Ha az objektum másolása nem triviális (azaz a fordító által generált másolás nem elegendő), akkor a megfelelő másolást is meg kell valósítani.

Kiindulási projekt, megoldás feltöltése

- **A megoldáshoz az előre kiadott osztályok módosítása szükséges lehet.**
 - Nem minden ZH esetében van kiindulási projekt.
- Feltöltéskor ezeket az osztályokat is fel kell tölteni és a módosításokat is pontozhatja a bíró!
- Egyes tesztesetekben a bíró módosított osztályt is használhat ezen kiinduló osztályok helyett, ezzel tesztelve a valóban helyes működést!

Zh alatt használható segédanyag

- A ZH során használható segédanyag elérhető bíróban.
 - <https://biro.inf.u-szeged.hu/kozos/native/>

Dinamikus memória követése

Ebben a feladatban lehetőség van egy speciális konstrukció használatára ami segít az allokált memória követésére. Amennyiben a `new` után egy `TRACK_INFO`-t írunk a `biro` rendszere ki tudja írni, hogy hol került allokálásra az adott memória terület ami nem lett felszabadítva (bizonyos tesztek esetén). Ezt az információt a `teszteset.stdout` kiterjesztésű fájljába írja bele.

Példa használat: `new TRACK_INFO Sword(...)`

Példa kimenet az `stdout` fájlban:

```
[ FAIL ] Memory error occured. Here is the map of the allocated memory addresses:
---| FAIL | 0xd7ee40 is not freed. filename: feladat.cpp:123 in function: <function>
---| FAIL | 0xd7eeb0 is not freed. filename: feladat.cpp:123 in function: <function>
---| OK  | 0xd7ef20 is nicely freed.
---| OK  | 0xd7ef50 is nicely freed. filename: feladat.cpp:133 in function: <function>
```

Figyelem! Amennyiben nem `biro-n` kerül fordításra a program, ilyen memória allokáció követés nem történik.

Kiindulás

A kiindulási feladatban kapott osztály módosítása vagy bővítése szükséges lehet!

TVProgram osztály

Az `TVProgram` egy tv-műsor osztályt reprezentál.

- A tv-műsornak van címe (`string`), amit a létrehozáskor meg kell adni.
- A `setTitle` metódussal be lehet állítani (meg lehet változtatni) a műsor címét. Az új címet paraméterben (`string`) kapja meg.
- Az `info` metódus "`TVProgram: <title>`" alakban adja vissza a tv-műsort, ahol a `<title>` a tv-műsor címe.

Movie osztály

A `Movie` osztály egy filmet reprezentál, ami a `TVProgram` osztályból származik.

- A filmnek van gyártási éve (előjeltelen egész szám), amit a létrehozáskor meg kell adni a film címe után.
- Az `info` metódus "`Movie: <title> (year: <year>)`" alakban adja vissza a film adatait, ahol a `<title>` a film címe és a `<year>` a gyártási éve.

Series osztály

A **Series** osztály egy filmsorozatot reprezentál, ami a **Movie** osztályból származik.

- A sorozatnak megadhatjuk, hogy hány epizódból áll (előjeltelen egész szám), amit a létrehozáskor címe és a gyártási éve után kell megadni.
- Az `info` metódus "`Series: <title> (year: <year>, episodes: <episodes>)`" alakban adja vissza a sorozatot, ahol a `<title>` a sorozat címe, a `<year>` a gyártási éve és a `<episodes>` azt mondja meg, hogy hány részből áll.

Feladatok

DailyPrograms osztály

A **DailyPrograms** osztály a napi tv-műsorokat tárolja.

- A tv-műsorokat (**TVProgram**) hozzáadási sorrendben kell eltárolni.
- Mivel egy tv-műsort többször is műsorra tűzhetnek, így az osztálynak csak akkor kell törölnie a tv-műsort, ha már senki sem használja.
- Az `add` metódus segítségével lehessen hozzáadni egy tv-műsort (**TVProgram**).
 - A paraméter típusa `shared_ptr<TVProgram>`.
 - Tárolja el az addigi tv-műsorok után.
 - Nem kell semmivel sem visszatérni.
- Legyen egy `get` metódus, ami paraméterben egy egész számot vár (`n`), és visszaadja az `n`-edik tv-műsorra mutató pointert.
 - Ha nincs ilyen elem, akkor adjon vissza `nullptr`-t.
- Az osztálynak legyen default konstruktora.
- Az osztály legyen másolható.
- Az osztály megfelelően kezelje és szükség esetén szabadítsa fel a memóriát.

Programs osztály

A **Programs** osztály a napi tv-műsorokat tárolja, azaz az adott naphoz elkészíthetünk egy napi tv-műsort (**DailyPrograms**), és ezeket tárolja el.

- **A napi programok egyediek**, mert előfordulhat, hogy az egyik napi programot módosítják műsorváltás miatt, de ez nem érintheti a többi napi programot.
- Az osztály a napokat egy egész számmal jelöli, és az adott naphoz képes napi tv-műsorokat (**DailyPrograms**) eltárolni.
 - Egy naphoz pontosan egy tv-műsor tartozhat.
 - A tárolás a napok szerint növekvő sorrendben történik.

- Az `add` metódussal lehessen új napi tv-műsort hozzáadni, ahol az első paraméter a nap, amihez tartozik (egész szám), a második paraméter a napi tv-műsor, aminek a típusa `unique_ptr<DailyPrograms>`.
 - Ha még nem volt az adott naphoz eltárolva napi tv-műsor, akkor tárolja el.
 - Ha már volt az adott naphoz napi tv-műsor eltárolva, akkor cserélje ki a paraméterben kapottra.
 - Nem kell semmivel sem visszatérnie.
- A `get` metódussal lehessen elkérni a napi tv-műsorra (`DailyPrograms`) mutató **pointert**.
 - A paramétere egy egész szám, és az ehhez tartozó napi tv-műsor (`DailyPrograms`) **pointert** kell visszaadni.
 - Ha nincs az adott naphoz műsor, akkor `nullptr`-rel kell visszatérni.
- Legyen egy kétparaméteres `get` metódus is, ahol az első paraméter (egész szám) megmondja, hogy melyik naphoz keressük a tv-műsort, a második paraméter (egész szám) meg azt határozza meg, hogy az adott napon belül melyik `TVProgram`-ot kell visszaadni.
 - A visszatérési típusa `TVProgram*`
 - Ha nincs ilyen nap, vagy nincs az adott napon elegendő tv-műsor, akkor `nullptr`-rel térjen vissza.
- Az osztálynak legyen default konstruktora.
- Valósítsd meg az osztály másoló konstruktorát!
- Az osztály megfelelően kezelje és szükség esetén szabadítsa fel a memóriát.

Algoritmusok

Az algoritmusokat a `"#if defined(MY_CAT) || !defined(TEST_BIRO)"` makrófeltétellel védett területen kell megvalósítani, különben a bíróba feltöltve fordítási hibát okoz a feladat többi részének.

`countCats` függvény

- Készíts egy `countCats` függvényt, ami egy `Cat`-eket tartalmazó `std::vector`-t és egy `string`-et kap paraméterben, és a `test::count_if` segítségével megszámolja, hogy hány olyan macska van a vektorban, amelyiknek a színe megegyezik a paraméterben kapott színnel és 10-nél kisebb a tömege. A kapott értékkel kell visszatérni.

`accumulateCats` függvény

- Valósítsd meg az `accumulateCats` függvényt, ami egy `Cat`-eket tartalmazó `std::vector`-t kap paraméterben, és a `test::accumulate` alkalmazásával összefűzi a vektorban található macskák színét vesszővel elválasztva, majd ezzel tér vissza (`std::string`). Például, ha a vektorban egy `fekete` és egy `fehér` macska van, akkor a visszatérési érték `"fekete,fehér,"` (idézőjelek nélkül) és az utolsó szín után is legyen vessző.